

ExcursionRepositories.mjs

This file performs the database work for the excursion Lambda.

Its main function is:

```
1 | setAlertRaisedAt()
```

It performs a DynamoDB:

```
1 | UpdateCommand
```

against:

```
1 | readingId
```

and sets:

```
1 | alertRaisedAt
```

The important condition

It uses a condition similar to:

```
1 | attribute_not_exists(  
2 |   alertRaisedAt  
3 | )
```

This is one of the most important pieces of the alert system.

SQS uses **at-least-once delivery**.

That means AWS might occasionally deliver the same message more than once.

Imagine:

```
1 | Message 1  
2 |   ↓  
3 | Lambda  
4 |   ↓  
5 | alertRaisedAt set  
6 |  
7 |  
8 | Same message again  
9 |   ↓  
10 | Lambda
```

Without protection, you could raise the alert twice.

Instead DynamoDB says:

```
1 | Does alertRaisedAt already exist?  
2 |  
3 | NO
```

```
4   ↓  
5   set it  
6   ↓  
7   result = raised  
8  
9  
10  YES  
11  ↓  
12  don't change it  
13  ↓  
14  result = duplicate
```

This is your **idempotency protection**.

Your earlier project design explicitly used this conditional `alertRaisedAt` update for duplicate protection.

Why does the repository do a `GetCommand` after failure?

A conditional DynamoDB update can fail because:

```
1 | alertRaisedAt already exists
```

But it might also fail because:

```
1 | reading doesn't exist  
2  
3 | branch doesn't match
```

Those aren't the same thing.

So after a conditional failure, the repository can retrieve the item:

```
1 | new GetCommand(...)
```

and inspect it.

If:

```
1 | item exists  
2 | AND  
3 | branch matches  
4 | AND  
5 | alertRaisedAt already exists
```

then:

```
1 | duplicate
```

Otherwise the error should continue upwards.

That's much safer than treating every conditional failure as a duplicate.